Sentinela Multimodal

Relatório de funcionamento e guia de execução no VS Code

Tech Challenge Fase 4 | Versão do projeto: 1.0.0 | 20 de outubro de 2026

1. O que é o projeto

O Sentinela é um protótipo acadêmico em Python para reunir dados de um paciente e gerar alertas sobre alterações que merecem revisão humana. Ele aceita sinais vitais, prescrições, movimentação, texto, áudio e observações extraídas de vídeo. A ideia é permitir que a equipe veja diferentes evidências na mesma sequência temporal.

A versão entregue funciona pelo terminal: não existe uma página web, aplicativo visual, servidor HTTP ou endereço localhost para abrir. O VS Code é o ambiente para editar o código, executar comandos, depurar e consultar os arquivos gerados. Os alertas aparecem no terminal e são registrados em um banco SQLite.

O núcleo local já foi executado. Azure e o modelo de vídeo possuem conectores implementados e testes simulados, mas ainda não foram validados com serviços reais e mídia clínica. Portanto, o projeto deve ser apresentado como protótipo de engenharia, sem alegar validação clínica ou diagnóstico automático.

2. Como os dados percorrem o sistema

O fluxo tem cinco etapas:

1. Receber dados estruturados ou extrair informações de um arquivo de áudio/vídeo.

2. Transformar a entrada em eventos com paciente, instante, modalidade e conteúdo.

3. Aplicar regras específicas para cada modalidade.

4. Correlacionar achados recentes do mesmo paciente.

5. Gravar os alertas e permitir que uma pessoa confirme que os revisou.

```
Áudio WAV -> energia e pausas -> transcrição Azure -> texto
Vídeo -> YOLOv8 Pose + ByteTrack -> observação postural
Sinais vitais / prescrições / movimentação -> eventos
                    |
                    v
           Evento normalizado JSONL
                    |
                    v
       Regras e histórico de cada paciente
                    |
                    v
         Fusão em janela de 5 minutos
                    |
                    v
        Console + SQLite + relatórios
                    |
                    v
        Revisão humana e confirmação
```

Áudio e vídeo são analisados em lote, a partir de arquivos. O monitor pode acompanhar novos eventos acrescentados a um arquivo, mas não captura câmera ou microfone ao vivo.

3. Tecnologias e dependências

Componente	Para que serve	Quando é necessário
Python 3.11 ou superior	Executa todo o programa	Sempre
Biblioteca padrão Python	JSON, estatística, WAV, CLI, datas e persistência	Demonstração e núcleo local
SQLite via sqlite3	Guarda eventos e alertas em arquivo .db	Monitoramento; sem servidor separado
Azure Speech SDK	Transcreve fala em português	Somente modo de áudio com --azure
Azure AI Text Analytics SDK	Sentimento e frases-chave	Somente modo --azure
Ultralytics e YOLOv8n Pose	Detectam pontos corporais no vídeo	Somente comando video
OpenCV	Lê os frames e o FPS do vídeo	Somente comando video
ByteTrack	Mantém rastros de pessoas entre frames	Acionado pelo extrator de vídeo
PyTorch e dependências transitivas	Executam o modelo YOLO	Instalados conforme dependências de vídeo
VS Code e extensão Python	Edição, seleção de ambiente e execução	Para trabalhar no editor
Git	Versionamento e publicação do código	Opcional para executar; necessário à entrega em repositório

O arquivo pyproject.toml declara os pacotes. As dependências principais estão vazias: a demonstração usa apenas a biblioteca padrão. Os extras são azure-cognitiveservices-speech >=1.40,<2, azure-ai-textanalytics >=5.3,<6, ultralytics >=8.3,<9 e opencv-python >=4.10,<5. Não há lockfile validado; instalações futuras podem resolver versões diferentes dentro dessas faixas.

O projeto não utiliza OpenAI API, ChatGPT, LLM, Supabase, Flask, FastAPI ou Streamlit. Não é necessário contratar esses serviços para executá-lo.

4. Organização dos arquivos

```
projeto-fase4/
  pyproject.toml          Metadados e dependências
  config.json             Parâmetros das regras
  .env.example            Nomes das variáveis Azure
  README.md               Orientação inicial
  sentinela/
    __main__.py           Entrada de python -m sentinela
    cli.py                Comandos e relatórios
    core.py               Regras, fusão e banco
    media.py              Áudio, Azure e vídeo
    demo.py               Dados sintéticos determinísticos
  tests/                  Testes automatizados
  docs/                   Relatório técnico e roteiro do vídeo
  evidencias/             Registros da verificação realizada
  resultados/             Saídas criadas ao executar
```

Não execute cli.py ou core.py isoladamente pelo botão de executar arquivo. Eles fazem parte de um pacote e usam importações relativas. O comando correto é python -m sentinela, a partir da pasta que contém config.json e pyproject.toml.

5. Preparar o VS Code no Windows

5.1. Extrair e abrir a pasta correta

Extraia projeto-fase4.zip. No VS Code, use Arquivo > Abrir Pasta e selecione a pasta projeto-fase4 que contém pyproject.toml, config.json e sentinela. Evite abrir apenas a pasta sentinela ou a pasta externa que contém o ZIP.

Uma cópia usada na demonstração desta conversa está em:

```
C:\Users\clini\Documents\Codex\2026-10-20\d\outputs\demonstracao\projeto-fase4
```

Instale Python 3.11 ou superior e a extensão Python da Microsoft no VS Code. Para depurar, use também a extensão Python Debugger, se ainda não estiver disponível. Abra Terminal > Novo Terminal e confira:

```
python --version
Get-Location
Get-ChildItem
```

Se python não for reconhecido, verifique se py --version funciona. O launcher py depende da instalação do Python. Nesse caso, use py -3.11 para criar o ambiente, se essa versão estiver instalada.

5.2. Criar um ambiente virtual

O ambiente virtual separa as bibliotecas do projeto das demais instalações do computador. No terminal aberto na raiz do projeto:

```
python -m venv .venv
```

Para executar sem depender da ativação do PowerShell, use diretamente o Python do ambiente:

```
.\.venv\Scripts\python.exe --version
.\.venv\Scripts\python.exe -m sentinela --help
```

Você também pode ativá-lo com o comando abaixo e passar a usar python nos exemplos seguintes:

```
.\.venv\Scripts\Activate.ps1
```

Se a ativação for bloqueada, continue usando .\.venv\Scripts\python.exe. Não é necessário alterar a política de execução do computador.

5.3. Selecionar o interpretador no editor

Pressione Ctrl+Shift+P, procure Python: Select Interpreter e selecione .venv\Scripts\python.exe. Se não aparecer, informe esse caminho manualmente. Abra um novo terminal depois da seleção. Confirme o executável em uso:

```
python -c "import sys; print(sys.executable)"
```

O resultado deve apontar para a pasta .venv do projeto. Selecionar o interpretador no editor evita que a análise do código e a depuração usem um Python diferente do terminal. Consulte a documentação oficial do VS Code indicada nas referências.

6. Primeira execução: demonstração local

Os comandos a seguir pressupõem o ambiente ativado e o terminal na raiz. Sem ativação, substitua python por .\.venv\Scripts\python.exe.

```
python -m sentinela demo --out resultados/minha-demo
```

Esse comando cria fixtures sintéticas, processa os eventos e gera os resultados. Não usa credenciais Azure, internet ou pesos YOLO. O resultado esperado numa pasta nova é de 20 eventos armazenados e 13 alertas, sem rejeições.

```
resultados/minha-demo/
  fixtures/events.jsonl
  fixtures/audio-sintetico.wav
  fixtures/transcricao.txt
  monitor.db
  alerts.json
  errors.json
  summary.json
  metrics.json
  report.md
```

Abra report.md no VS Code e pressione Ctrl+Shift+V para visualizar o Markdown. Abra summary.json para os totais. O banco é binário; não tente editá-lo como texto. Para uma execução independente, escolha outro nome de pasta, como resultados/demo-02. Reutilizar uma pasta mantém o banco e os estados de confirmação anteriores.

6.1. Entender os resultados

Arquivo	O que consultar
summary.json	Quantidade lida nesta execução, total armazenado, alertas por código e rejeições
alerts.json	Evidência, gravidade, paciente, instante e estado de cada alerta
errors.json	Linhas rejeitadas e motivo
report.md	Tabela de alertas para leitura
metrics.json	Comparação entre rótulos sintéticos e códigos previstos
monitor.db	Registro persistente de eventos e alertas

events_read_this_run conta linhas aceitas nesta execução, inclusive repetições já conhecidas. stored_events conta eventos únicos gravados. Por isso os dois números não significam a mesma coisa.

7. O que acontece na demonstração

O paciente fictício começa com doze medidas para construir o histórico. Depois, recebe a sequência abaixo:

Instante do cenário, em UTC	Evento sintético	Resposta
12:02:00	FC 145 bpm e SpO2 88%	4 alertas: limites e desvios do histórico
12:02:05	Termos críticos e pausas de áudio	2 achados e 1 fusão

Instante do cenário, em UTC	Evento sintético	Resposta
12:02:10	Observação de tronco inclinado	1 achado postural e 1 fusão
12:02:20	Dose demonstrativa de 10 para 30	1 alteração de dose e 1 fusão
12:02:30	Imobilidade por 2100 segundos	1 achado e 1 fusão

O total é de 13 alertas, mas isso não significa 13 problemas clínicos independentes. Um evento pode disparar mais de uma regra e também um alerta de fusão. A data dos fixtures é 01/01/2026 e não representa uma consulta real.

Na demonstração, a observação de vídeo já vem pronta como dado sintético. O WAV contém tom e silêncio; o texto é fornecido separadamente. Nenhum desses fixtures comprova execução real do Azure ou inferência real do YOLO.

8. Consultar e confirmar alertas

```
python -m sentinela alerts --db resultados/minha-demo/monitor.db
```

Copie o campo id de um alerta e substitua ID_DO_ALERTA no comando:

```
python -m sentinela ack ID_DO_ALERTA --db resultados/minha-demo/monitor.db
```

O estado muda de pending_review para acknowledged. Isso registra uma confirmação de leitura/revisão, não um diagnóstico ou resolução da ocorrência. O protótipo não identifica o profissional responsável.

alerts lê o banco atualizado. Já alerts.json e report.md são retratos da última execução de run ou demo; o comando ack não os atualiza. Para renovar essas exportações, reprocesse o mesmo arquivo na mesma pasta:

```
python -m sentinela run resultados/minha-demo/fixtures/events.jsonl --out resultados/minha-demo
```

Os IDs repetidos serão ignorados e os estados persistidos serão preservados.

9. Como cada análise funciona

9.1. Sinais vitais

O programa aceita frequência cardíaca, saturação e pressão sistólica. Compara as medidas com os limites de config.json e com o histórico anterior do mesmo paciente. Não analisa pressão diastólica, ECG ou imagens de exames nesta versão.

O histórico guarda até 30 valores normais. Após pelo menos 8 valores, calcula mediana e MAD, uma medida robusta da dispersão. A fórmula é:

```
escore = |valor - mediana| / max(1,4826 x MAD, piso)
```

Escore maior ou igual a 4 gera alerta relativo. O piso impede divisão por zero quando a série é constante. Valores sinalizados como anormais não entram no histórico. Essa escolha reduz contaminação, mas pode manter uma baseline antiga quando o estado do paciente muda permanentemente.

9.2. Prescrições

Compara a dose diária do mesmo medicamento, para o mesmo paciente, com a dose anterior. Dobrar ou reduzir à metade dispara a regra. Mudança de unidade gera revisão separada. Não converte mg para g, não interpreta esquemas de administração e não verifica interações ou adequação terapêutica. O produtor deve fornecer a dose diária total de forma consistente.

9.3. Texto e negação

O léxico local procura falta de ar, dor no peito, desmaio e dificuldade para respirar. Normaliza acentos e considera algumas palavras de negação nas quatro palavras anteriores, dentro de uma oração. Isso permite ignorar exemplos como "nega falta de ar", mas não compreende toda a linguagem clínica.

9.4. Áudio

O extrator calcula energia RMS em janelas de 30 ms. Valores abaixo de 0,015 são considerados silêncio. Retorna duração, proporção de silêncio, maior pausa e energia média. Razão de silêncio superior a 0,55 pode gerar alerta. O método não separa a fala do profissional da fala do paciente e não distingue pausa, ambiente silencioso e falha de gravação. Não diagnostica fadiga, disartria ou dificuldade respiratória.

9.5. Vídeo

YOLOv8n Pose estima pontos do corpo e ByteTrack associa rastros entre frames. O extrator usa ombros e quadris com confiança mínima de 0,5. Inclinação do tronco superior a 40 graus por pelo menos 2 segundos gera observação. A perda de confiança ou rastreamento interrompe a persistência; um novo episódio pode gerar outro achado.

O recorte é fisioterapia com câmera frontal fixa. Uma inclinação pode ser parte correta do exercício. O programa não conhece o protocolo terapêutico, não reconhece falhas cirúrgicas e não identifica pessoas. Todos os rastros são associados ao paciente passado no comando: use apenas o paciente de interesse no vídeo.

9.6. Movimentação e fusão

Movimentação recebe uma duração de imobilidade já calculada por sensor ou outro produtor. Não há conector de sensor físico entregue. Duração de pelo menos 1800 segundos dispara a regra.

A fusão reúne modalidades com achados nos últimos 300 segundos do mesmo paciente. Duas ou mais modalidades geram um alerta adicional com as evidências. É uma regra determinística, sem probabilidade de risco; modalidades correlacionadas não equivalem a confirmações clínicas independentes.

10. Alterar as configurações

Parâmetro	Valor inicial	Significado
baseline_window	30	Tamanho máximo do histórico normal
baseline_min	8	Medidas necessárias para escore robusto
robust_z	4,0	Limiar do escore de desvio
fusion_window_seconds	300	Janela de correlação
dose_ratio	2,0	Razão para mudança de dose
silence_ratio	0,55	Razão de silêncio para alerta
immobility_seconds	1800	Duração de imobilidade para alerta
video_trunk_angle_degrees	40	Limiar de inclinação
video_persistence_seconds	2	Persistência exigida

Os limites iniciais são FC de 45 a 130 bpm, SpO2 de 92 a 100% e pressão sistólica de 90 a 180 mmHg. São valores demonstrativos do código, sem validação como protocolo médico. Os limites absolutos são inclusivos; só valores fora deles disparam essa regra.

Mantenha JSON válido, com ponto decimal e sem comentários. Para testar outra configuração, copie o arquivo, modifique os valores e use --config antes do subcomando:

```
python -m sentinela --config config-teste.json demo --out resultados/config-teste
```

Não reinterprete um banco antigo com configuração alterada: a reconstrução do estado usará as regras novas, enquanto os alertas antigos continuarão armazenados. Prefira uma pasta nova por configuração. O projeto não possui versionamento de regras no banco nem validador completo de configuração; preserve valores coerentes e positivos.

11. Processar áudio local

Use WAV PCM mono de 16 bits. MP3 ou gravação estéreo não são aceitos diretamente. O modo local exige um arquivo TXT com a transcrição; ele não converte fala em texto automaticamente.

Para copiar comandos longos, prefira a versão Markdown deste guia. Cada comando abaixo deve ser enviado como uma única linha no terminal.

```
python -m sentinela audio resultados/minha-demo/fixtures/audio-sintetico.wav --transcript
  resultados/minha-demo/fixtures/transcricao.txt --patient PAC-DEMO --timestamp
  2026-01-01T12:02:05Z --id audio-local-001 --out resultados/audio-local.jsonl
```

O comando audio extrai características e cria um evento; a detecção final ocorre ao consumir esse evento:

```
python -m sentinela run resultados/audio-local.jsonl --out resultados/audio-monitor
```

No teste efetuado com o WAV sintético, a duração foi de 8 segundos, a razão de silêncio aproximadamente 0,7491 e a maior pausa de 6 segundos. A origem da transcrição é user_supplied. No modo local, sentimento aparece como not_evaluated.

12. Habilitar Azure

Instale o extra Azure dentro do ambiente virtual:

```
python -m pip install -e ".[azure]"
```

É necessário possuir assinatura e recursos compatíveis de Speech e Language. Copie do portal as credenciais de cada recurso. Speech usa chave e região; Language usa chave e endpoint. São configurações separadas. No terminal da sessão:

```
$env:AZURE_SPEECH_KEY="CHAVE_DO_SPEECH"
$env:AZURE_SPEECH_REGION="REGIAO_DO_RECURSO"
$env:AZURE_LANGUAGE_KEY="CHAVE_DO_LANGUAGE"
$env:AZURE_LANGUAGE_ENDPOINT="https://SEU_RECURSO.cognitiveservices.azure.com/"
```

Os valores acima são marcadores: devem ser substituídos pelos reais. .env.example documenta os nomes, mas o aplicativo não carrega arquivos .env automaticamente. Definir variáveis neste terminal não as define em todos os terminais ou sessões de depuração. Não grave chaves em código, prints ou commits.

Prepare dados-reais/consulta.wav com fala autorizada e execute:

```
python -m sentinela audio dados-reais/consulta.wav --azure --patient PAC-DEMO --timestamp
  2026-01-01T12:00:00Z --id consulta-001 --out resultados/consulta.jsonl
python -m sentinela run resultados/consulta.jsonl --out resultados/consulta-monitor
```

O Speech usa reconhecimento contínuo pt-BR, reúne segmentos e espera o fim da sessão. Há timeout de 600 segundos na espera; erros não são trocados por texto fictício. O texto reconhecido é enviado ao Language para sentimento e frases-chave. A regra de termos críticos continua local; sentimento não aumenta gravidade. Neste protótipo, transcrições com mais de 5000 caracteres são rejeitadas pelo conector Language: é necessário segmentá-las externamente, preservando IDs e contexto.

Serviços podem gerar cobrança conforme os recursos e o uso da sua assinatura; consulte as condições atuais no portal antes da execução. Nenhuma conta, recurso ou integração Azure real foi configurada nesta entrega.

13. Habilitar vídeo

```
python -m pip install -e ".[video]"
```

Para ambos os extras, use python -m pip install -e ".[azure,video]". As dependências de vídeo podem exigir downloads grandes. O modelo pode baixar yolov8n-pose.pt na primeira execução, e o rastreador pode solicitar dependência adicional. Não há benchmark de hardware para esta entrega; CPU pode ser suficiente para experimentação, mas o tempo depende do vídeo e do computador. GPU compatível pode acelerar, conforme a instalação do PyTorch.

```
python -m sentinela video dados-reais/fisioterapia.mp4 --patient PAC-DEMO --timestamp
  2026-01-01T12:00:00Z --id fisio-001 --out resultados/video.jsonl
python -m sentinela run resultados/video.jsonl --out resultados/video-monitor
```

Use --weights CAMINHO para escolher um arquivo de pesos compatível. O timestamp representa o início da gravação; os eventos recebem esse instante mais o deslocamento do achado. A saída contém video.jsonl e video.jsonl.report.json. Mesmo sem achados, o relatório registra frames, FPS e duração; o JSONL pode ficar vazio. Não é gerado vídeo anotado com esqueleto nesta versão. Para distribuição comercial, verifique a licença aplicável da Ultralytics e dos pesos.

14. Criar eventos e acompanhar novas entradas

JSONL significa um objeto JSON completo por linha, em UTF-8. Não é um array JSON e não contém vírgulas entre linhas. Exemplo de um evento:

```
{"id":"vital-001","patient_id":"PAC-
  DEMO","timestamp":"2026-01-
  01T12:00:00Z","modality":"vitals","data":{"heart_rate":145,"spo2":88}}
```

14.1. Regras do contrato

- id: identificador globalmente único e imutável; reutilizar o ID descarta a repetição, mesmo se você mudou seu conteúdo.

- patient_id: identificador consistente do paciente; é a chave para isolar histórico e fusão.

- timestamp: ISO 8601 com fuso, como Z ou -03:00. Z é UTC. Não altere apenas o sufixo para converter horário: preserve o instante real.

- modality: vitals, prescription, movement, text, audio ou video.

- data: objeto com os campos da modalidade, documentados no README.

Os eventos devem vir em ordem cronológica por paciente. Entradas atrasadas são rejeitadas. Não há conversão automática de unidades, medicamentos equivalentes ou identificação de paciente pela imagem.

14.2. Monitorar um arquivo em crescimento

Crie eventos.jsonl vazio e execute:

```
python -m sentinela run eventos.jsonl --follow --out resultados/ao-vivo
```

Um produtor separado deve acrescentar linhas completas terminadas em quebra de linha. O monitor consulta novas entradas a cada 250 ms. Use Ctrl+C para encerrar de forma controlada e exportar os relatórios. Durante a execução, os alertas já são persistidos no banco; os arquivos de resumo são exportados ao sair do laço. O monitor não acompanha rotação ou truncamento do arquivo.

--interval 0.2 adiciona espera de 0,2 segundo entre linhas e ajuda a demonstrar o fluxo; não é uma reprodução fiel dos intervalos dos timestamps.

14.3. Combinar as modalidades

Para demonstrar fusão com mídia real, use o mesmo patient_id, timestamps alinhados e IDs distintos; reúna os eventos num JSONL ordenado cronologicamente. Alimentar pastas de banco separadas não produz fusão entre elas. Se você processar um arquivo antigo depois de eventos mais novos do mesmo paciente, ele será rejeitado; planeje a ordenação antes da ingestão.

15. Testes e depuração no VS Code

```
python -m unittest discover -s tests -v
```

A verificação registrada executou 17 testes com sucesso no Python 3.12.14. Inclui isolamento de pacientes, validade de entradas, expiração da fusão, idempotência, persistência, confirmação, baseline e conectores simulados. Os testes de Azure não chamam os serviços e o teste de vídeo não executa o modelo real.

No painel Testing do VS Code, configure unittest com diretório tests e padrão test*.py. Para depurar a demonstração, crie .vscode/launch.json com o conteúdo abaixo, selecione a configuração e pressione F5:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Sentinela: demonstracao",
      "type": "debugpy",
      "request": "launch",
      "module": "sentinela",
      "cwd": "${workspaceFolder}",
      "args": ["demo", "--out", "resultados/debug"],
      "console": "integratedTerminal",
      "justMyCode": true
    }
  ]
}
```

Esse é um exemplo de configuração para você criar; não é um arquivo já incluído no ZIP original. Coloque um breakpoint em Engine.process, dentro de core.py, e observe event, hits, history e recent. A depuração depende do interpretador selecionado no VS Code. Confira a documentação oficial nas referências para detalhes atuais do editor.

Para recalcular a avaliação sintética:

```
python -m sentinela evaluate resultados/minha-demo/fixtures/events.jsonl --out
  resultados/avaliacao.json
```

evaluate exige expected_codes em cada evento e serve aos fixtures rotulados. Não use esse comando em dados reais sem rótulos. O F1 de 1,0 observado nos fixtures é um teste de regressão construído a partir das próprias regras, não uma estimativa de desempenho clínico.

16. Problemas comuns e soluções

Sintoma	O que conferir
python não é reconhecido	Instalação do Python e PATH; verificar launcher py
No module named sentinela	Estar na raiz extraída; selecionar o interpretador correto
config.json não encontrado	Executar na raiz ou passar --config com caminho correto antes do comando
Attempted relative import	Usar python -m sentinela, em vez de executar cli.py diretamente
No module named azure, cv2 ou ultralytics	Instalar o extra no mesmo Python em uso com python -m pip
Activate.ps1 bloqueado	Executar .venv\Scripts\python.exe diretamente
Chave Azure ausente	Definir as quatro variáveis no ambiente que executa o programa
Falha de autenticação ou rede Azure	Conferir recurso, chave, região, endpoint, permissões e conectividade
WAV recusado	Garantir PCM mono de 16 bits; não apenas renomear MP3 para WAV
Nenhum frame ou FPS inválido	Conferir caminho, arquivo e decodificação pelo OpenCV
Nenhum alerta no vídeo	Conferir confiança, enquadramento, persistência e relatório; ausência de alerta não prova normalidade
ID repetido não altera resultado	Usar novo ID e banco de teste novo para um evento modificado
Evento fora de ordem	Ordenar por instante antes da ingestão; conferir UTC e fuso
ack não modifica report.md	Consultar alerts no banco ou renovar exportação com run
Banco bloqueado	Manter um consumidor e evitar gravadores concorrentes
Texto longo demais no Azure Language	Segmentar a transcrição; o limite local é de 5000 caracteres

Se houver rejeições, consulte errors.json. O retorno 0 indica conclusão sem rejeições; 1 indica falha ou eventos rejeitados. No PowerShell, consulte $LASTEXITCODE logo após o comando. Algumas falhas inesperadas de SDKs, configuração inválida ou arquivos corrompidos ainda podem exigir análise técnica; o protótipo não oferece recuperação completa de todas as exceções.

17. Banco, manutenção e segurança dos dados

O SQLite fica no arquivo monitor.db de cada pasta de resultados. Há tabelas de eventos e alertas. Você não precisa instalar PostgreSQL, MySQL ou um servidor de banco. Ao reiniciar, o programa lê os eventos para reconstruir histórico e fusão; essa estratégia pode ficar lenta conforme o volume cresce.

Para uma nova experiência, use uma pasta de saída nova. Para backup, encerre o processo antes de copiar o banco e seus relatórios. Os dados ficam em claro e podem conter informações sensíveis; o protótipo não possui autenticação, criptografia de banco ou trilha de auditoria por profissional.

Não envie .env, chaves, .venv, dados clínicos ou bancos de pacientes ao Git. O .gitignore ajuda a excluir alguns arquivos, mas não substitui revisar o conteúdo. A pasta evidencias contém somente dados sintéticos; parte dos relatórios JSON pode ser incluída no versionamento deliberadamente. Usar dados reais exige definição

institucional de acesso, tratamento, retenção e validação.

Não existem rotinas de prescrição automática, integração com prontuário, disparo de SMS/email, detecção de ausência de sensor ou garantia de disponibilidade. A memória de IDs e o replay do histórico também não foram dimensionados para grande escala. Essas são evoluções possíveis, não funcionalidades entregues.

18. Publicação e entrega acadêmica

O ZIP contém o código, documentação e evidências locais. Não é, por si só, um repositório Git remoto publicado. Para iniciar versionamento, dentro da raiz e com Git instalado:

```
git init
git add .
git status
```

Revise os arquivos preparados antes do commit. Depois:

```
git commit -m "Adiciona prototipo Sentinela multimodal"
```

Crie um repositório na conta da equipe e siga as instruções do provedor para configurar o remoto e enviar. Não foi criado remoto nesta entrega.

O relatório técnico e o roteiro da apresentação estão em docs. Para concluir os entregáveis do desafio, ainda será necessário demonstrar chamadas Azure reais, processar vídeo de fisioterapia real autorizado, registrar essas evidências, publicar o código e gravar um vídeo de até 15 minutos no YouTube ou Vimeo. A gravação não foi produzida ou publicada nesta conversa.

Ao apresentar, diferencie claramente os 13 alertas sintéticos, o áudio de tom/silêncio, os testes simulados e qualquer execução real feita posteriormente. Não anuncie acurácia clínica de 100% com base nos fixtures.

19. Referências e base deste relatório

Este guia foi conferido contra pyproject.toml, config.json, cli.py, core.py, media.py e os registros de testes do projeto entregue. As referências abaixo apoiam as instruções de software e serviços, não os limiares clínicos demonstrativos:

- Python no VS Code

- Ambientes Python no VS Code

- Depuração Python no VS Code

- Azure SpeechConfig

- Azure SpeechRecognizer

- Azure AI Text Analytics para Python

- Ultralytics: rastreamento de objetos

- Ultralytics: estimação de pose

Os nomes de menus e a disponibilidade de serviços podem mudar; use as páginas oficiais para conferir a interface instalada. A primeira experiência recomendada é executar demo, consultar report.md, confirmar um alerta e rodar a suíte de testes antes de instalar as integrações opcionais.